The Picky Builder: Decision Trees with LEGO

Machine Learning — Master in AI — Decision Trees

Ricardo Tavares (2026133913) André Leão (2023210870) José Antunes (2023211288)

Code: `https://github.com/migasantunes/decision-trees-lego`

1 The model

A decision tree classifies an observation by asking a sequence of yes/no questions, each a threshold on a single feature. A leaf predicts the majority class of the training observations that reached it, and the class fraction in the leaf is the predicted probability.

Training is greedy and top-down. At each node the algorithm tries every feature and threshold and keeps the split with the highest *information gain*, the drop in entropy from the parent node to the size-weighted children:

$$H(S) = -\sum_i p_i \log_2 p_i, \qquad Gain(S, \mathrm{A}) = H(S) - \sum_{v \in \{\text{yes, no}\}} \frac{|S_v|}{|S|} H(S_v),$$

where p_i is the share of class i in node S ($H = 0$ for a pure node, $H = 1$ for a 50/50 binary node) and S_v are the two children. Splitting continues until leaves are pure or a stopping rule applies (`max_depth`, `min_samples_leaf`). We used scikit-learn's `DecisionTreeClassifier` with `criterion="entropy"`, the information-gain criterion of ID3.

Because a fully grown tree keeps splitting until every leaf is pure, it can carve a region around any single training point. This is the source of both of the behaviours we study: overfitting to label noise and instability under small changes to the training set.

2 Experimental design

Data. The demonstration is built around a "picky builder" who sorts LEGO bricks into *Accepted* or *Rejected*. Each synthetic brick has three features matching the physical kits used in the class activity: `stud_rows` (short side, 1–2), `stud_cols` (long side, 2–10; shapes 1×2, 1×3, 1×4, 1×6, 1×10, 2×2, 2×3, 2×4, 2×6, 2×8) and `colour` (8 colours, one-hot encoded so that a split reads "is it yellow?" instead of an arbitrary ordering of colours). This gives 80 possible brick types. Orientation is fixed (rows is always the short side), so a 2×4 and a 4×2 are the same brick.

Target. The builder's hidden rule is *Accepted* $\iff$ (yellow and `stud_cols` ≥ 4) or (not yellow **and** `stud_rows` $= 2$). It uses all three features and a depth-3 tree can represent it exactly, which we checked before any experiment by fitting a depth-3 tree to all 300 clean bricks (accuracy 1.0, all three features used). The acceptance rate is 53.7%, so the classes are close to balanced.

Protocol (fixed for all experiments). $N = 300$ bricks, with shape and colour drawn uniformly and independently; stratified 70/30 split (210 train / 90 test) stratified on the true label; seed 42; entropy criterion. The one-hot encoder is fitted on training data only. Hyperparameters are chosen only with 5-fold cross-validation on the training set; the test set is used once, for the final score. Label noise flips exactly 15% of the labels, chosen at random, on the whole dataset, so the test labels (the "stickers") are noisy too.

Metrics. Accuracy is the main metric because the classes are balanced; macro-F1 was reported alongside in Exp. 1 and was within 0.002 of accuracy. Under noise we report two test scores: against the *stickers* and against the *truth*. The best achievable accuracy against the stickers is 1−(share of flipped test labels), 0.811 for seed 42. We also count the *brick types predicted against the rule* out of all 80, which measures what the tree has learned independently of the test sample.

Experiments. Each experiment changes one aspect relative to the previous one. The code is split into three notebooks, one per experiment, sharing the data generator and plotting helpers in `src/`: `dev_a.ipynb` (data checks and Exp. 1), `dev_b.ipynb` (Exp. 2 and the stability check) and `dev_c.ipynb` (Exp. 3). The predictions below were written before running and are kept unedited in these notebooks.

- **Exp. 1, favourable case:** clean labels, no depth limit. *Prediction:* almost perfect train and test accuracy; a colour split near the root; all three features used.[1]
- **Exp. 2, controlled failure:** the only change is 15% label noise. *Prediction:* training accuracy close to, but not exactly, 100% (identical bricks with different stickers cannot be separated); lower test accuracy, with a ceiling of about 85% against the stickers; many more leaves, created to isolate noisy bricks.
- **Stability check:** the Exp. 2 setting, but each of $B = 100$ trees is trained on a bootstrap resample of the 210 training bricks (same draws for clean and noisy labels). *Prediction:* the root stays `stud_rows` in almost every tree; clean trees always have 6 leaves and agree on all 80 types; noisy trees have about 25–35 leaves with a wide range and lower agreement.

[1] The prediction originally named red as the special colour; it was changed to yellow to match the physical kits before running. On clean data `max_depth=None` and `max_depth=3` give the same tree.

- **Exp. 3, attempted improvement:** same noisy data as Exp. 2, with `max_depth` $\in \{2, 3, 4, 5, 6, \text{None}\}$ and `min_samples_leaf` $\in \{1, 2, 5, 8, 10\}$ chosen by 5-fold `GridSearchCV`, repeated over 5 seeds. *Prediction:* test accuracy better than Exp. 2 but still below Exp. 1, because of the noise.

3 Results

Table 1: Main results, seed 42 (same 300 bricks and same split in all three experiments). Multi-seed rows: mean ± std of test accuracy against the truth (Exp. 1–2: 10 seeds, 0–9; Exp. 3: 5 seeds, 0, 42, 123, 456, 789).

	Exp. 1 (clean)	Exp. 2 (15% noise)	Exp. 3 (noise + CV)
Hyperparameters	no limit	no limit	`max_depth=2`, `min_samples_leaf=1`
Training accuracy	1.000	0.900	0.852
Test accuracy vs stickers	0.989	0.778 (ceiling 0.811)	0.778
Test accuracy vs truth	0.989	0.900	0.967
Leaves / depth	6 / 3	48 / 12	4 / 2
Brick types against the rule (of 80)	1	10	5
Test vs truth, multiple seeds	1.000 ± 0.000	0.916 ± 0.044	0.973 ± 0.015
Test vs stickers, multiple seeds	1.000 ± 0.000	0.794 ± 0.042	0.818 ± 0.036

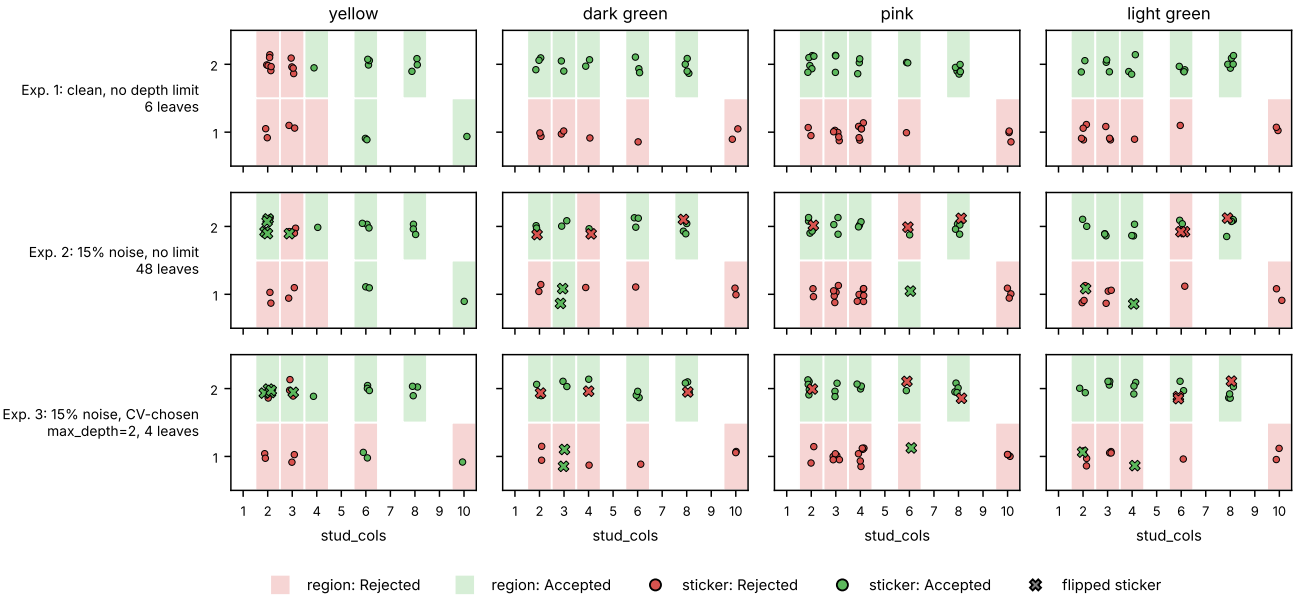


Figure 1: Decision regions (`stud_cols` × `stud_rows`) for four of the eight colours, seed 42, with the training bricks on top (small jitter for display). Exp. 2 grows islands around flipped stickers (e.g. dark green 1×3, light green 1×4). Exp. 3 removes the islands but, at depth 2, can no longer express the yellow exception. Full eight-colour plots are in the notebooks.

3.1 Exp. 1: the tree recovers the rule

The tree stopped by itself at depth 3 with 6 pure leaves, training accuracy 1.000 and test accuracy 0.989 (one error in 90). Its structure matches the rule, with one threshold off: for 1-row yellow bricks it learned `stud_cols` > 4.5 instead of ≥ 4. The training set had 1-row yellow bricks with 2, 3, 6 and 10 columns but none with 4, so any threshold between 3 and 6 separates them and scikit-learn takes the midpoint. The single test error is exactly a yellow 1×4: a tree only knows the boundary where it has data near it.

Prediction check: the accuracy and the use of all three features were right, but the root was `stud_rows`, not a colour split. Splitting on rows first gives the larger information gain, because the "2 rows" condition decides the label for the 7 non-yellow colours at once. Following one test brick: a yellow 2×6 goes `stud_rows` $> 1.5 \rightarrow$ yellow $\rightarrow$ `stud_cols` > 3.5 and lands in a leaf where 7 of 7 training bricks are Accepted, so it is predicted Accepted.

3.2 Exp. 2: the tree memorises the noise

With 15% of the stickers flipped, the unrestricted tree grew from 6 to 48 leaves and from depth 3 to 12. Test accuracy fell to 0.778 against the stickers (ceiling 0.811) and 0.900 against the truth, and 10 of the 80 brick types are now predicted against the rule (Table 1, Figure 1). Of the 28 noisy training bricks, 15 were memorised in their own regions; 9 of the 10 wrong brick types contain a flipped training brick, and the tenth is the yellow 1×4

boundary from Exp. 1. One flipped light green 1×4 needs 7 questions instead of 2 and ends in a leaf holding only itself, so every future light green 1×4 will be accepted against the rule. Feature importances spread over all colours and `colour_yellow` fell from 0.22 to 0.04: the tree spends its questions on noise and loses track of which colour matters. The failure appeared in all 10 seeds (48.5 ± 4.0 leaves).

Prediction check: test accuracy and tree size were right. Training accuracy was only partly right: we expected close to 1.000 and got 0.900, because most brick types have several copies, so a flipped brick is usually outvoted by its correct twins in the same leaf (13 of the 28 noisy bricks lost that vote; 19 brick types carry conflicting stickers).

3.3 Stability: the root never moves, everything below it does

Across 100 bootstrap trees the root was `stud_rows` every time, with clean and with noisy labels: the information-gain gap at the top is too large for a resample to close. Below the root the picture changes (Figure 2). Noisy trees had 35.6 leaves on average (range 24–44, depth 11.0), and for 55 of the 80 brick types at least one tree disagreed with the majority (mean agreement 0.911). Clean trees were not perfectly stable either, contrary to our prediction: 6 brick types were not unanimous (mean agreement 0.992) and 10 trees had only 4 or 5 leaves. All 6 are yellow types with at most 2 training copies; a brick with one copy is missing from about 37% of the resamples, and without it the threshold moves. The noisy-tree size (35.6) was just above our predicted 25–35. A deep tree is therefore unstable: with clean labels the instability is confined to rare bricks near the boundary, with noise it spreads across most of the map.

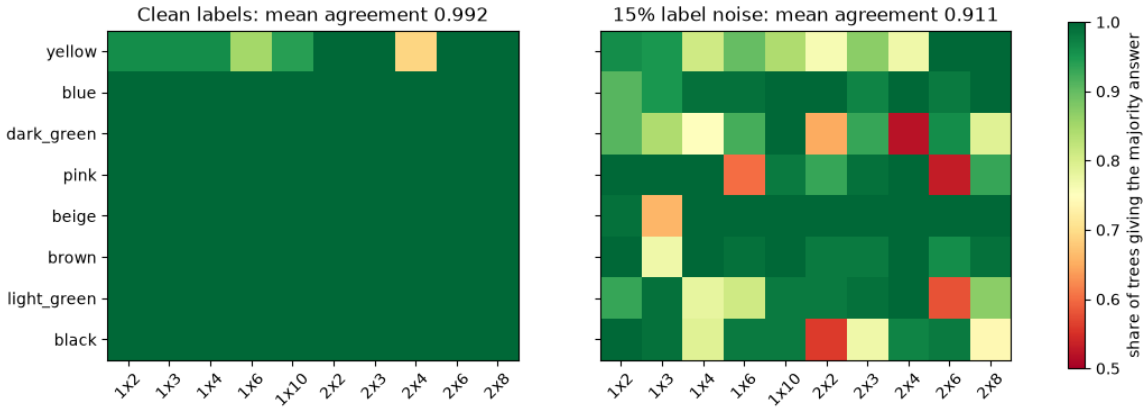


Figure 2: Agreement of 100 bootstrap trees for each of the 80 brick types (rows: colours; columns: shapes). Green: all trees give the same answer; red: close to a coin flip.

3.4 Exp. 3: limiting the tree helps, but removes part of the rule

Cross-validation always chose `min_samples_leaf=1` and a limited depth (2–5 over the five seeds). Over five seeds, test accuracy against the truth was 0.973 ± 0.015, compared with 0.916 ± 0.044 for the unrestricted tree of Exp. 2 (ten seeds), and against the stickers 0.818 ± 0.036 compared with 0.794 ± 0.042, close to the ceiling. At seed 42 the score against the truth rose from 0.900 to 0.967, while the score against the stickers did not change (0.778). The gain against the truth is larger than the spread across seeds; against the stickers it is within the spread, because 15–20% of the test stickers are wrong whatever the model does. Training accuracy dropped to 0.84–0.87: a limited tree no longer fits the flipped stickers, which is the intended effect. Our prediction, better than Exp. 2 and worse than Exp. 1, was right.

The remaining limitation shows at seed 42. Cross-validation accuracy was 0.852 for every depth from 2 to 5, and `GridSearchCV` returned the first of the tied settings, `max_depth=2`. The resulting tree splits on `stud_rows` and then on two colours whose children predict the same class, so in effect it asks a single question: 2 rows → Accepted, 1 row → Rejected. The islands are gone, but the yellow exception is lost: 5 brick types, all yellow, are now systematically wrong (Figure 1). Yellow covers only one colour in eight, and with 15% noise, cross-validation on 210 bricks cannot tell the true exception apart from noise. The fix exchanges overfitting for underfitting of a rare but real part of the rule.

4 Limitations

- Synthetic data with one known, axis-aligned rule; the label noise is random and symmetric, whereas real noise is often systematic.
- The "test vs truth" score is only possible with synthetic data; in practice only the noisy stickers are available, and against them the improvement of Exp. 3 is within the seed-to-seed spread.
- The size of the failure depends on how many copies of each brick type exist: with only 80 types and 210 training bricks, part of the noise is outvoted, so a larger dataset would weaken it further.

- Changing the seed also changes the test set, so the multi-seed standard deviation mixes model instability with test sampling; the bootstrap check isolates the first.

5 Three practical lessons

1. **Never trust a fully grown tree, and look at what pruning removed.** Limit `max_depth` or `min_samples_leaf` and choose them by cross-validation on training data only. Then inspect the chosen tree: when CV scores tie, the simplest setting can erase a rare but genuine pattern, as it did with the yellow exception.
2. **Read train/test gaps, tree size and importances together.** A large gap with many small leaves signals overfitting, and the feature importances of an overfitted tree are misleading: noise spread importance over colours the rule never uses. Trees need no feature scaling, but categorical features should be one-hot encoded so that each split is a meaningful question.
3. **One tree is fragile below the root.** Small changes in the training data redraw the lower branches, and rare examples near a boundary are where it breaks first. Check a tree across seeds or bootstrap resamples, and when stability matters, average many trees (bagging, random forests).

6 Contributions

Ricardo Tavares: Experiment 3 (attempted improvement): hyperparameter search over `max_depth` and `min_samples_leaf` with `GridSearchCV`, evaluation over five seeds and analysis (`dev_c.ipynb`); report structure and integration; Exp. 3 presentation slides.

André Leão: Experiment 2 (controlled failure): design, prediction, implementation and analysis (`dev_b.ipynb`); redesign of the protocol so that Exp. 2 changes only the label noise, with the same bricks and split as Exp. 1; bootstrap stability check; physical LEGO kit for the class activity (pieces, special colour, noisy kits, mystery bricks), with `src/data.py` aligned to the kit; Lego race and Exp. 2 presentation slides.

José Antunes: repository setup; synthetic data generator, encoder and data checks (`src/data.py`); plotting helpers (`src/plots.py`); Experiment 1 (favourable case) including the root-to-leaf trace (`dev_a.ipynb`); project plan; presentation template and the model and Exp. 1 slides.

7 AI use declaration

Tools: Claude (Anthropic), GitHub Copilot, Gemini, Google Search.
What they supported: the project plan and division of tasks; explanations of decision-tree concepts (entropy, information gain, overfitting, cross-validation) used to prepare the experiments and the Q&A; code review and suggestions, code for the plotting helpers.